

ADS- Practice Question

- 1] Declare an Array of 5 subjects. Accept subject marks from user. Calculate the avg or percentage
- 2] Write a program to accept number n from user & check if its even or odd.
- 3] Write a program to print prime number betⁿ 1 & 50
- 4] Write a menu driven program to print weekend & weekdays.
- 5] Write a program to accept number from user & to check if its palindrom or not.

Codes

```
1] → #include <stdio.h>
int main()
{
    int i, marks[5], sum=0;
    printf("Enter marks of 5 subjects:");
    for (i=0; i<5; i++)
    {
        scanf("%d", &marks[i]);
    }
    for (i=0; i<5; i++) {
        sum += marks[i];
    }
    float Avg = sum/5;
    printf("Average = %.2f", Avg);
    return 0;
}
```

2] #include <stdio.h>

int main()

{ int n;

printf("Enter a number n=");

scanf("%d", &n);

if((n % 2) == 0)

{ printf("Even Number");
}

else {

printf("Odd Number");
}

return 0;

}

3] #include <stdio.h>

int main()

{ int num, i, count=0;

printf("Prime numbers from 1 to 50 are: \n");

for (num=1; num <= 50; num++)

{

for (i=2; i < num; i++)

{

if (num % i == 0)

{

count++;

break;

}

}

}

```
if (count == 0 && i != 1)
    printf ("%d\n", num);
    count = 0;
}
return 0;
}
```

4] #include <stdio.h>

```
int main()
```

```
{ int num;
```

```
    printf ("Enter a number:");
```

```
    scanf ("%d", &num);
```

```
    switch (num){
```

```
        case 1:
```

```
            printf ("Monday In Weekday");
```

```
            break;
```

```
        case 2:
```

```
            printf ("Tuesday In Weekday");
```

```
            break;
```

```
        case 3:
```

```
            printf ("Wednesday In Weekday");
```

```
            break;
```

```
        case 4:
```

```
            printf ("Thursday In Weekday");
```

```
            break;
```

case 5:

```
printf("Friday In weekday");
```

```
break;
```

case 6:

```
printf("Saturday In weekend");
```

```
break;
```

case 7:

```
printf("Sunday In weekend");
```

```
break;
```

```
default;
```

```
printf("Invalid");
```

```
break;
```

```
}
```

```
return 0;
```

```
}
```

5] #include <stdio.h>

```
int main() {
```

```
int num, reversed=0, remainder, original;
```

```
printf("Enter an integer: ");
```

```
scanf("%d", &num);
```

```
original = num;
```

```
while (num != 0)
```

```
    remainder = num % 10;
```



```

{
    remainder = num % 10;
    reversed = reversed * 10 + remainder;
    num = num / 10;
}
if (original == reversed)
{
    printf("%d is a palindrome", original);
}
else {
    printf("%d is not a palindrome", original);
}
return 0;
}

```

→ Output:

Enter an integer: 67

67 is not a palindrome number.

Outputs:

1] Enter Marks of 5 subjects:

96 78 98 95 74

Average Marks : 88.00

2] Enter a Number: 50.

Odd number.

3] Prime Number betⁿ 1 to 50:

1 2 3 5 7 11 13 17 19 23

24 29 31 37 41 43 47

4] Enter a Number: 6.

Saturday

Weekend/Holiday.

~~Null~~

Experiment-2

Q- Write a C code to find avg of 10 numbers.

→ #include <stdio.h>

int main() {

int numbers[10];

int sum = 0;

float average;

printf("Enter 10 numbers: \n");

for (int i = 0; i < 10; i++) {

scanf("%d", &numbers[i]);

sum += numbers[i];

}

average = sum / 10.0;

printf("Average = %.2f\n", average);

return 0;

}

→ Output:

Enter 10 numbers: 1

2

3

4

5

6

7

8

9

10

Average = 5.50

Q Print this following code : *

##

####

→ #include <stdio.h>

int main() {

for (int i=1; i<=4; i++) {

for (int j=1; j<=i; j++) {

if (i%2 == 1) {

printf("*");

} else {

printf("#");

}

}

printf("\n");

}

return 0;

}

→ Output:

*

##

####

Q] Find First Repeating element of an array:

→ #include <stdio.h>

```
int main () {
```

```
    int n, i, j, arr[100];
```

```
    printf("Enter the number of elements:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[i] == arr[j]) {
```

```
                printf("First Repeating element is: %d \n",
```

```
                arr[i]);
```

```
                return 0;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("No Repeating elements found. \n");
```

```
    return 0;
```

```
}
```

→ Output:

Enter the number of elements: 3

Enter elements: 2

2

1

First repeating element is: 2.

Q] Find the greatest & smallest element in an array.

```
→ #include <stdio.h>
int main() {
    int n, i;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: \n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int smallest = arr[0];
    int greatest = arr[0];
    for (i = 1; i < n; i++) {
        if (arr[i] < smallest)
            smallest = arr[i];
        if (arr[i] > greatest)
            greatest = arr[i];
    }
    printf("Smallest element: %d \n", smallest);
    printf("Greatest element: %d \n", greatest);
    return 0;
}
```

→ Output:

Enter the number of elements: 4

Enter 4 elements: 2, 1, 3, 7

Smallest element: 1

Greatest element: 7

```

5] #include <stdio.h>
int main()
{
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: \n", n);
    for(int i=0; i<n; i++){
        if (i % 2 == 0){
            arr[i] = arr[i] * arr[i];
        }
    }
    printf("Array after squaring swapping elements at  
odd positions: \n");
    for(int i=0; i<n; i++){
        printf("%d", arr[i]);
    }
    printf("\n");
    return 0;
}

```

→ Output:

Enter the number of elements: 3

Enter 3 elements:

1

2

9.

Array after ^{squaring} ~~swap~~ elements at odd position
1 2 81

* Practice:

17

*

* *

* * *

* * * *

→ #include <stdio.h>

int main()

{

int i, j;

for(i=1; i<=4; i++)

{

for(j=1; j<=4-i; j++)

{

printf(" ");

}

for(j=1; j<=i; j++)

{

printf("* ");

}

printf("\n");

}

return 0;

}

```

2] * * * *
   * * *
     * *
      *

```

→ #include <stdio.h>

```
int main() {
```

```
    int i, j;
```

```
    for (i = 4; i >= 1; i--)
```

```
    {
```

```
        for (j = 1; j <= 4 - i; j++)
```

```
        {
```

```
            printf(" ");
```

```
        }
```

```
        for (j = 1; j <= 2 * i - 1; j++)
```

```
        {
```

```
            printf(" * ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
3] *
```

```
  # #
```

```
 $ $ $
```

```
? ? ? ?
```


→ #include <stdio.h>

int main() {

int i, j;

for (i = 1; i <= 4; i++) {

for (j = 1; j <= i; j++)

{

if (i == 1)

printf("*");

else if (i == 2)

printf("# -");

else if (i == 3)

printf("\$ -");

else if (i == 4)

printf("? -");

}

printf("\n");

}

return 0;

}

4] 1

1 2

1 2 3

1 2 3 4

1 2 3 4 5


```

→ #include <stdio.h>

int main()
{
    int i, j;
    for (i = 1; i <= 5; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d - ", j);
        }
        printf("\n");
    }

    return 0;
}

```

Q5

```

1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

```

```

→ #include <stdio.h>

int main() {
    int i, j;
    for (i = 1; i <= 5; i++) {
        for (j = 1; j <= i; j++) {
            printf("%d - ", j);
        }
        printf("\n");
    }
    return 0;
}

```

Experiment-2

- ① Search Data Using linear search consider following list to perform linear search:
56, 36, 89, 57, 1, 0, 67, 59

→ #include <stdio.h>

int main()

{ printf("Enter 8 Elements : \n");

for(int i=0; i<8; i++)

{

scanf("%d", &a[i]);

}

printf("Enter Search Key: ");

scanf("%d", &k);

for(int i=0; i<8; i++)

{

if(a[i] == k)

{

printf("Element %d Found at %d ", a[i], i+1);

return 0;

}

}

printf("Element not Found");

return 0;

}

1] O/P:

Enter Elements:

36

89

57

1

0

67

59

56

Enter Search Key : 1

Element found at 4

2] O/P

~~Enter~~ Enter Elements:

36

89

57

1

0

67

59

56

Enter Search Key: 55

Element not found.

Q Search Sama data of linear Search using binary Search Method.

```
→ #include <stdio.h>
int main() {
    int a[100], n, k, l, m, h;
    printf("Enter Number of Array:");
    scanf("%d", &n);
    printf("Enter %d Sorted Elements: \n", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    printf("Enter the Key to Search:");
    scanf("%d", &k);
    l = 0;
    h = n - 1;
    while (l <= h) {
        m = (l + h) / 2;
        if (a[m] == k) {
            printf("Key found at index %d \n", m + 1);
            return 0;
        }
        else if (a[m] < k) {
            l = m + 1;
        }
        else {
            h = m - 1;
        }
    }
    printf("Key not found in Array \n");
    return 0;
}
```

1] O/p:
Enter Number of Array: 8
Enter 8 Sorted Elements:

0
1
36
56
57
59
67
89

Key For Enter the Key to Search: 1
Element found at index 2

2] O/p:
Enter Number of Array: 8
Enter 8 Sorted Elements:

0
1
36
56
57
59
67
89

Enter the Key to Search: 55
Element Not found

~~O/P:~~

Enter Number of Array : 8

Enter 8 Sorted Elements:

0

1

Q3] Compare Linear & binary Search.

→

Linear Search

Binary Search.

- | | |
|--|--|
| • It works on both sorted & unsorted arrays. | • it only works on sorted array. |
| • Time Complexity in (n) | • Time Complexity in $\log(n)$ |
| • It is used for small & unsorted data set. | • It is used for large & sorted dataset. |
| • Simple to Implement | • Complex to implement. |
| • Algorithm Type is sequential. | • Algorithm type is divide & conquer. |

Q4] State limitations of Linear Search in terms of time complexity.

→ it has limitations in terms of time complexity especially for large data every element in array to determine if it is present or not, which makes it slower. The binary data is sorted.

Abhishek
01/12/21

Practice Question

Q- Write a Program to print array in reverse order.

→ #include <stdio.h>

int main()

int arr1[10], arr2[10], i;

printf("Enter 10 elements of array: \n");

for (i = 0; i < 10; i++) {

scanf("%d", &arr1[i]);

}

for (i = 0; i < 10; i++) {

arr2[i] = arr1[9-i];

}

printf("The Given Array is: ");

for (i = 0; i < 10; i++) {

printf("%d", arr1[i]);

}

printf("In The Reversed Array is: ");

for (i = 0; i < 10; i++) {

printf("%d", arr2[i]);

}

return 0;

}

O/P :

Enter 10 Elements of the Array:

1 2 3 4 5 6 7 8 9 10

Given Array is : 1 2 3 4 5 6 7 8 9 10

Reversed Array is : 10 9 8 7 6 5 4 3 2 1

Q- Write a Program to Count number of Duplicate elements present in Array.

```
→ #include <stdio.h>
int main() {
    int arr[100], n, i, j, count=0;
    printf("Enter number of Elements: ");
    scanf("%d", &n);
    printf("Enter elements: \n");
    for(i=0; i<n; i++) {
        scanf("%d", &arr[i]);
    }
    for(i=0; i<n; i++) {
        for(j=i+1; j<n; j++) {
            if(arr[i] == arr[j]) {
                count++;
                break;
            }
        }
    }
    printf("Total Number of Duplicate Number = %d\n", count);
    return 0;
}
```

→ O/p:

Enter Number of elements: 6

Enter Elements:

2 3 2 3 4 5

Total number of duplicate number = 2

Q Write a program to print unique element from array.

```
→ #include <stdio.h>
int main()
{
    int arr[100], n, i, j, count;
    printf("Enter number of elements : ");
    scanf("%d", &n);

    printf("Enter elements: \n");
    for(i=0; i<n; i++){
        scanf("%d", &arr[i]);
    }

    printf("Unique elements are: \n");
    for(i=0; i<n; i++){
        count=0;
        for(j=0; j<n; j++){
            if(i != j && arr[i] == arr[j]){
                count++;
                break;
            }
        }
        if(count==0){
            printf("%d", arr[i]);
        }
    }

    return 0;
}
```

O/P:

Enter number of elements : 5

Enter elements:

5

4

2

2

5

Unique elements are : 4

Q- Write a Program to Separate Even & Odd from Array.

```
→ #include <stdio.h>
int main() {
    int arr[100], even[100], odd[100];
    int n, i, e=0, o=0;
    printf("Enter Number of elements:");
    scanf("%d", &n);
    printf("Enter Elements: \n");
    for(i=0; i<n; i++) {
        scanf("%d", &arr[i]);
        if(arr[i] % 2 == 0) {
            even[e] = arr[i];
            e++;
        } else {
            odd[o] = arr[i];
            o++;
        }
    }
    printf("Even Numbers are:");
    for(i=0; i<e; i++) {
        printf("%d", even[i]);
    }
    printf("Odd Number are: \n");
    for(i=0; i<o; i++) {
        printf("%d", odd[i]);
    }
    return 0;
}
```


Enter Number of elements: 4

Elements:

2

4

9

6

Even Number are: 2, 4, 6

Odd Number are: 9

Q Write a program to print second smallest number.

→ #include <stdio.h>

int main()

{
int arr[100], n, i, j, temp;

printf("Enter Number of elements: ");

scanf("%d", &n);

printf("Enter Elements");

for(i=0; i<n; i++){

scanf("%d", &arr[i]);

}

for(i=0; i<n-1; i++){

for(j=i+1; j<n; j++){

if(arr[i] > arr[j]){

temp = arr[i];

arr[i] = arr[j];

arr[j] = temp;

}}}

for(i=1; i<n; i++){

if(arr[i] != arr[0]){

printf("Second Smallest Number: %d", arr[i]);

return 0;

}

O/P =
 Enter Number of element 4.
 Enter element 4.

- 1
- 2
- 3
- 4

Second Smallest Element is : 2.

Q

→ #include <stdio.h>

int main() {

char str[100];

int i=0, count=0;

printf("Enter a string:");

fgets(str, size of(str), stdin);

while (str[i] != '\0') {

if (str[i] == ' ' && str[i+1] != ' ' &&
str[i+1] != '\0') {

count++;

}

i++

}

if (str[0] != '\n') {

count++;

}

printf("Total number of words = %d\n",
count);

return 0;

}

O/P:

Enter a String: Neel Sonawane

Total number of words = 2

✓

Experiment-3

Q- Write a C program to Sort Array Using Bubble Sort.

→ #include <stdio.h>

int main()

{
int arr[100], n, i, j, temp;

printf("Enter Number of Elements: ");

scanf("%d", &n);

printf("Enter %d elements: \n", n);

for (i=0; i<n; i++) {

scanf("%d", &arr[i]);

}

for (i=0; i<n-1; i++) {

for (j=0; j<n-i-1; j++) {

if (arr[j] > arr[j+1]) {

temp = arr[j];

arr[j] = arr[j+1];

arr[j+1] = temp;

}

}

}

printf("Sorted array: \n");

for (i=0; i<n; i++) {

printf("%d", arr[i]);

}

return 0;

O/P:

Enter Number of elements: 5

Enter 5 elements:

4

8

16

7

Sorted array:

4 7 8 16

Q- Selection Sort:

```
→ #include <stdio.h>
int main() {
    int arr[100], n, i, j, min, temp;
    printf("Enter Number of elements:");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i=0; i<n; i++) {
        scanf("%d", &arr[i]);
    }
    for (i=0; i<n-1; i++) {
        min=i;
        for (j=i+1; j<n; j++) {
            if (arr[j] < arr[min])
            {
                min=j;
            }
        }
        temp=arr[i];
        arr[i]=arr[min];
        arr[min]=temp;
    }
    printf("Sorted array: \n");
    for (i=0; i<n; i++) {
        printf("%d", arr[i]);
    }
    return 0;
}
```

→ O/P:

Enter Number of element : 5.

Enter 5 elements :

8

6

9

7

8

Sorted Array:

9

7

8

6

3

~~if (arr[i] > arr[j])~~

~~arr[i] = arr[j]~~

arr[i] = temp

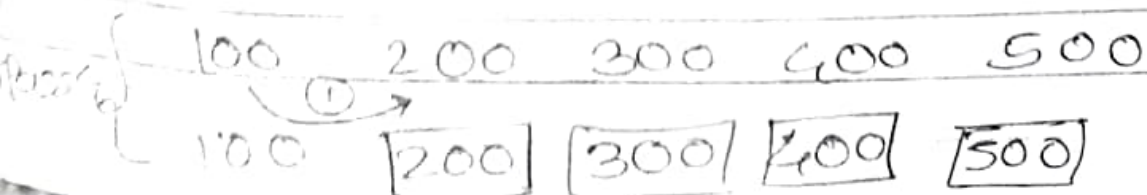
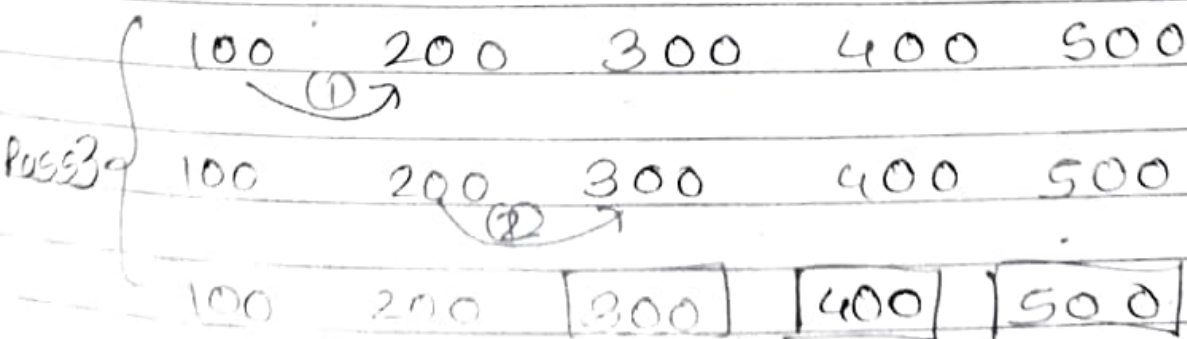
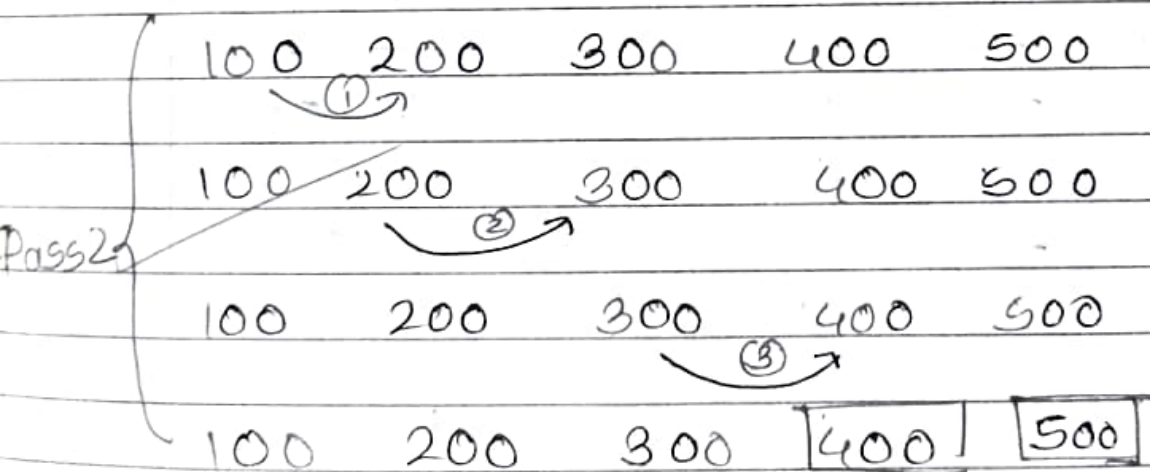
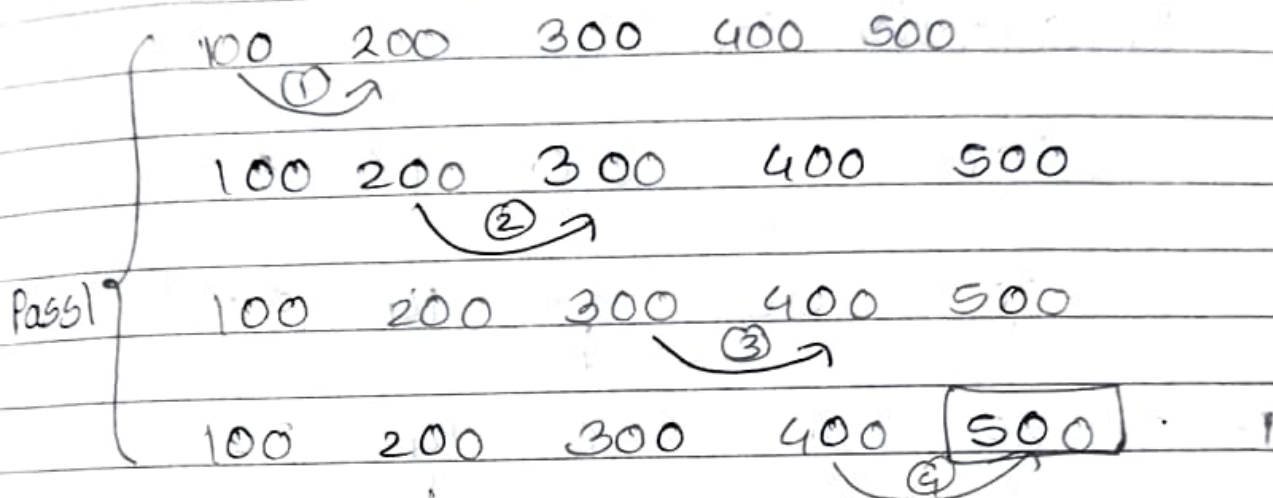
arr[j] = arr[i]

temp = arr[j]

Print("Sorted array")

3] Bubble Sort:

100 200 300 400 500



Total comparison.
 $4 + 3 + 2 + 1 = 10.$

4] selection sort:

Pass 1

500	-20	30	14	50.
-20	500	30	14	50.
-20	500	30	14	50
-20	500	30	14	50
-20	500	30	14	50

Pass 2

-20	500	30	14	50
-20	30	500	14	50.
-20	14	500	30	50.
-20	14	500	30	50

Pass 3

-20	14	500	30	50
-20	14	30	500	50.

Pass 4

-20 14 30 500 50

-20 14 30 50 500

~~17/10~~

Experiment-4

Q- Sort element in ascending order using insertion sort.

→ #include <stdio.h>

int main()

{

int a[50], n, i, j, temp;

printf("Enter no. of elements: ");

scanf("%d", &n);

printf("Enter %d elements", n);

for (i = 0; i < n; i++)

{

scanf("%d", &a[i]);

}

for (i = 1; i < n; i++)

{

for (j = 0; j < i; j++)

{

if (a[i] < a[j])

{

temp = a[i];

for (int k = i - 1; k > j; k--)

{

a[j + 1] = a[k];

}

a[j] = temp;

break;

}


```

}
printf("After Pass %d: ", i);
for (p = 0; p < n; p++)
{
    printf("%d ", arr[p]);
}
printf("\n");
}
printf("Sorted array: \n");
for (i = 0; i < n; i++)
{
    printf("%d \n", arr[i]);
}
return 0;
}

```

O/p:

Enter the Number of elements: 5

Enter array elements: 9

8

7

6

5

After pass 1: 8 9 7 6 5

After pass 2: 7 8 9 6 5

After pass 3: 6 7 8 9 5

After pass 4: 5 6 7 8 9

Sorted Array: 5 6 7 8 9

Q2] Sort element using Radix Sort in ascending order.

→ #include <stdio.h>

int main(){

int n;

printf("Enter Number of Elements : ");

scanf("%d", &n);

int arr[n], max=0;

printf("Enter %d elements :", n);

for(int i=0; i<n; i++)

{

scanf("%d", &arr[i]);

if (arr[i] > max) max = arr[i];

}

printf("Original Array:\n");

for (int i=0; i<n; i++)

{ printf("%d", arr[i]);

printf("\n\n");

}

int exp, pass=0;

int output[n];

int count[10];

for (exp=1; max/exp > 0; exp*=10){

for (int i=0; i<10; i++) count[i]=0;

for (int i=0; i<n; i++) count[arr[i]/exp % 10]++;

```

For (int i=1; i<10; i++) count[i] += count[i-1];
For (int i=n; i>1; i--)
    output[--count[arr[i]/exp)/10] = arr[i];
For (int i=0; i<n; i++) arr[i] = output[i];

printf("After Pass: \n", ++pass);
For (int i=0; i<n; i++) printf("%d", arr[i]);
printf("\n");
}

printf("Ascending Sorted Array: \n");
For (int i=0; i<n; i++)
    printf("%d", arr[i]);
return 0;
}

```

→ O/P:

Enter number of elements: 6

Enter 6 elements: 741

852

963

123

456

789.

Original Array: 741, 852, 963, 123, 456, 789.

After Pass 1: 741, 852, 963, 123, 456, 789

After Pass 2: 123, 741, 852, 456, 963, 789.

After Pass 3: 123, 456, 741, 789, 852, 963

Sorted Array: 123, 456, 741, 789, 852, 963

Q3] What is output of the insertion sort after 2nd iteration for following.

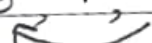
7, 3, 1, 9, 4, 8, 6.

1st Iteration

7, 3, 1, 9, 4, 8, 6


3, 7, 1, 9, 4, 8, 6.

2nd Iteration

3, 7, 1, 9, 4, 8, 6


~~1, 3, 7, 9, 4, 8, 6~~ → Output After 2nd Iteration

Q4] Solve using Radix Sort:

1st Pass 100, 225, 390, 4130, 956, 99, 5431.

Element/bucket	0	1	2	3	4	5	6	7	8	9
100	100									
225					225					
390	390									
4130	4130									
956						956				
99										99
5431		5431								

Bucket Element	0	1	2	3	4	5	6	7	8	9
100	100									
390										390
4130				4130						
5431				5431						
225			225							
956						956				
99.										99

$\Rightarrow 100, 225, 4130, 5431, 956, 390, 99.$

Bucket Element	0	1	2	3	4	5	6	7	8	9
100	100									
225			225							
4130		4130								
5431				5431						
956										956
390				390						
99.	99									

$\Rightarrow 99, 100, 4130, 225, 390, 5431, 956.$

	0	1	2	3	4	5	6	7	8	9
99	99									
100	100				4130					
4130					4130	5431				
225	225									
390	390									
5431						5431				
956	956									

Sorted Array = 99, 100, 225, 390, 4130, 5431, 956

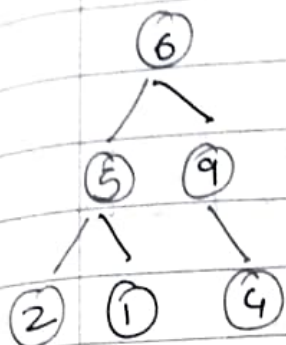
Q5] Sort using heap sort

i 6, 5, 9, 2, 1, 4.

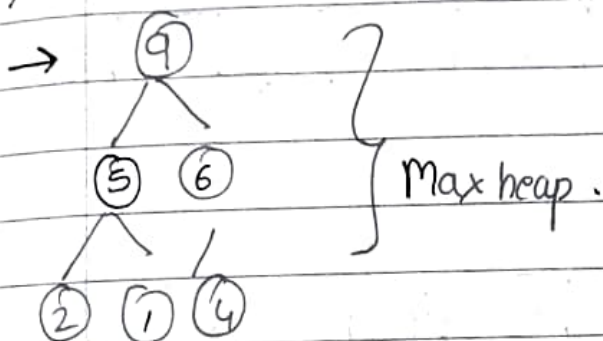
$$= n/2 - 1$$

$$= 6/2 - 1$$

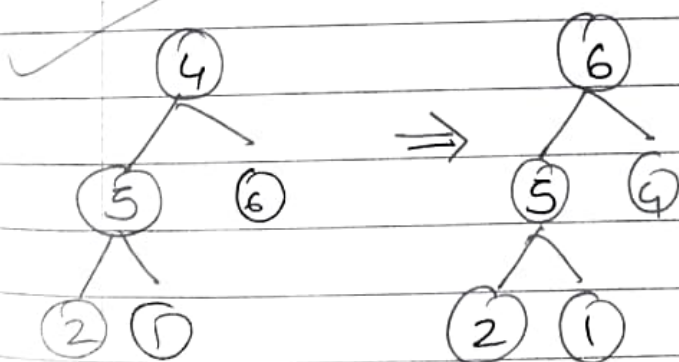
$$= 2.$$



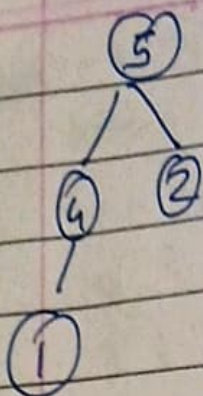
x



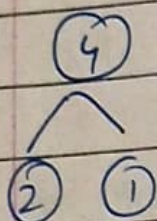
→ 4, 3, 6, 2, 1, 9



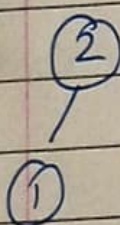
5, 4, 2, 1, 6, 9



$\Rightarrow 4, 2, 1, \boxed{6}, \boxed{6}, \boxed{9}$



$\Rightarrow 2, 1, \boxed{4}, \boxed{5}, \boxed{6}, \boxed{9}$



$\Rightarrow 1, \boxed{2}, \boxed{4}, \boxed{5}, \boxed{6}, \boxed{9}$

$\Rightarrow \text{1}, \boxed{2}, \boxed{4}, \boxed{5}, \boxed{6}, \boxed{9}$

~~17/10~~

Experiment - 5

Q- Write a Menu driven Program for linked list.

→ #include <stdio.h>

#include <stdlib.h>

struct Node {

int data;

struct Node* next;

};

struct Node* head = NULL;

void Insert At Beginning(),

void Insert At Position();

void Insert At End();

void Delete First();

void Delete End();

void delete at position();

void display();

void create list();

int main()

{

int choice

while (1) {

printf("1. Create list");

PAGE No.
 DATE
 printf("\n 2. Insert at Beginning");
 printf("\n 3. Insert at End");
 printf("\n 4. Insert at Position");
 printf("\n 5. delete First Node");
 printf("\n 6. Delete from Position");
 printf("\n 7. Delete last Node");
 printf("\n 8. Display list");
 printf("\n 9. Exit");
 printf("\n Enter Your Choice:");
 scanf("%d", &choice);

switch(choice){

case 1:

create list();

break;

case 2:

Insert At Beginning();

break;

case 3:

Insert At End();

break;

case 4:

Insert at position();

break;

case 5:

Delete First();

break;

case6:

deleteEnd();

break;

case7:

~~delete~~ delete At Position();

break;

case8:

~~deleted~~ display();

break;

case9:

exit(0)

~~default:~~

printf("Invalid choice!");

}

}

return 0;

}

void CreateList()

{

if(head != NULL)

{

printf("List Already Exists:");

return;

}

struct Node* newNode = (struct Node*)

malloc(sizeof(struct Node));

printf("Enter data for first Node:");

```
naonode → next = NULL;
head = new Node;
printf("In list created with node: %d",
naonode → data);
}
```

```
void insertAtBeginning();
{
```

```
    struct Node* newNode (struct Node*)
    malloc (size of (struct Node));
    printf("Enter data to insert at
    beginning");
    scanf("%d", &newNode → data);
    newNode → next = head;
    head = newNode;
    printf("In Inserted %d at the beginning",
    newNode → data);
}
```

```
void insertAtEnd() {
```

```
    struct Node* newNode = (struct Node*) malloc (size
    of (struct Node));
    printf("Enter data to insert At End: ");
    scanf("%d", &newnode → data);
    newNode → next = NULL;
```

```
if (head == NULL) {
```

```
    head = newNode;
```

```
    printf("Inserted %d as first node. In ", newNode
    → data);
```



```
} else {
```

```
    struct Node* temp = head;
```

```
    while (temp -> next != NULL) {
```

```
        temp = temp -> next;
```

```
    }
```

```
    temp -> next = newNode;
```

```
    printf("Inserted %d at the end.\n",  
        newNode -> data);
```

```
}
```

```
}
```

```
void insertAtPosition() {
```

```
    int pos;
```

```
    printf("Enter position to insert at (1-based):
```

```
    scanf("%d", &pos);
```

```
    if (pos < 1) {
```

```
        printf("Position must be >= 1! \n");
```

```
        return;
```

```
    }
```

```
struct Node* temp = head;
```

```
struct Node* newNode = (struct Node*) malloc (sizeof  
(struct Node));
```

```
printf("Enter Data for new node:");
```

```
scanf("%d", &newNode -> data);
```

```
if (pos == 1) {
```

```
    newNode -> next = head;
```

```
    head = newNode;
```

```
    printf("Inserted %d at position 1.\n", newNode  
        -> data);
```

```
}
```



```

struct Node* temp = head;
for (int i = 1; i < pos - 1; i++) {
    temp = temp->next;
    if (temp == NULL) {
        printf("Position %d is out of bounds!\n", pos);
        free(newNode);
        return 0;
    }
}

```

```

if (temp == NULL) {
    printf("Position %d is out of bounds!\n", pos);
    free(newNode);
    return;
}

```

```

newNode->next = temp->next;
temp->next = newNode;
printf("Inserted %d at position %d.\n", newNode->data, pos);
}

```

```

void deleteFirst() {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }
}

```

```

struct Node* temp = head;
head = head->next;
printf("Deleted first node with data %d.\n", temp->data);

```

```
free(temp);
```

```
}
```

```
void deletelast(){
```

```
    if (head == NULL){
```

```
        printf("List is Empty!");
```

```
        return 0;
```

```
    }
```

```
    if (head->next == NULL){
```

```
        printf("Deleted last Node with data %d.\n",  
            head->data);
```

```
        free(head);
```

```
        head = NULL;
```

```
        return;
```

```
    }
```

```
    struct Node* temp = head;
```

```
    while (temp->next->next != NULL){
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("Deleted last Node with data %d.\n",  
        temp->next->data);
```

```
    free(temp->next);
```

```
    temp->next = NULL;
```

```
}
```

```
void deleteAtPosition(){
```

```
    int pos;
```

```
    printf("Enter position to delete from (1-based  
        : ");
```

```
    scanf("%d", &pos);
```


PAGE NO.
 DATE
 if (pos < 1 || head == NULL) {
 printf("Invalid position or empty list!\n");
 return 0;
}

if (pos == 1) {
 deleteFirst();
 return 0;
}

struct Node* temp = head;
for (int i = 1; i < pos - 1; i++) {
 temp = temp->next;
 if (temp == NULL || temp->next == NULL) {
 printf("Position %d is out of bounds!\n", pos);
 return 0;
 }
}

if (temp->next == NULL) {
 printf("Position %d is out of bounds!\n", pos);
 return 0;
}

struct Node* nodeToDelete = temp->next;
temp->next = nodeToDelete->next;
printf("Deleted node at position %d with data %d.\n", pos, nodeToDelete->data);
free(nodeToDelete);

void Display () {

if (head == NULL) {
 printf("List is Empty.\n");
 return 0;
}


```

struct Node* temp = head;
printf("list = ");
while (temp != NULL) {
    printf("%d-> ", temp->data);
    temp = temp->next;
}
printf("NULL\n");
}

```

→ Output.

--- SINGLY LINKED LIST ---

1. Create list
2. ~~Create~~ Insert at Beginning
3. Insert at End
4. Insert at Position
5. Delete First Node
6. Delete last Node
7. Delete From position
8. Display list
9. Exit

Enter your choice: 1

Enter data for first node: 45

list created with node: 45

)

Enter your choice: 3

Enter data to insert at end: 75

Inserted a 75 at the end.

Enter your choice: 8

list: 45 $\rightarrow$ 75 $\rightarrow$ NULL.

Q Difference betⁿ Array & Linked list.

Array

Linked list

- | | |
|--------------------------------|---------------------------------|
| • Resizing not possible | Resizing is possible |
| • Memory allocations is static | Memory can change |
| • stored in contiguous memory | stored in non-contiguous memory |
| • $O(1)$ | $O(n)$ |

Q Basic Terminology of linked list.

a) Node: Basic building block of linked list
It contains data & next

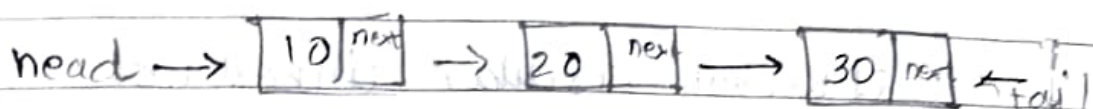


Node

b) head: the head is a reference or pointer to first node of list NULL or none.

c) tail: tail node where next = NULL

d] Next: Each node has a next reference to the following node.



e] NULL Pointer: Special value that indicates that pointer does not have object/memory.

f) Empty linked list: singly linked list with node in it.

~~Nil~~
n/p

Head $\rightarrow$ null.

Doubly Linked List.

```
#include <stdio.h>
#include <stdlib.h>

void create();
void display();
void insert_begin();
void insert_end();
void insert_position();

struct node {
    int info;
    struct node* next;
    struct node* prev;
};

struct node* start = NULL;

int main() {
    int choice;
    while (1) {
        printf("\n MENU ");
        printf("\n 1. CREATE");
        printf("\n 2. DISPLAY");
        printf("\n 3. INSERT_BEGIN");
        printf("\n 4. INSERT_END");
        printf("\n 5. INSERT_POSITION");
        printf("\n 6. EXIT");
        printf("\n Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                create();
                break;
            case 2:
                display();
                break;
            case 3:
                insert_begin();
                break;
            case 4:
                insert_end();
                break;
            case 5:
                insert_position();
                break;
            case 6:
                exit(0);
                break;
        }
    }
}
```

```

        insert_end();
        break;
    case 5:
        insert_position();
        break;
    case 6:
        printf("Exiting program.\n");
        exit(0);
    default:
        printf("Invalid choice, please try again.\n");
    }
}
return 0;
}

```

```

void create() {
    struct node* temp;
    struct node* ptr;

    temp = (struct node*)malloc(sizeof(struct node));
    if (temp == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    printf("\n Enter data: ");
    scanf("%d", &temp->info);
    temp->next = NULL;
    temp->prev = NULL;

    if (start == NULL) {
        start = temp;
    } else {
        ptr = start;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = temp;
        temp->prev = ptr;
    }

    printf("Node created and added at the end.\n");
}

void display() {

```

```

struct node* ptr;
if (start == NULL) {
    printf("\n Empty List\n");
    return;
}
ptr = start;
printf("\n List elements are: ");
while (ptr != NULL) {
    printf("%d ", ptr->info);
    ptr = ptr->next;
}
printf("\n");
}

void insert_begin() {
    struct node* temp;
    temp = (struct node*)malloc(sizeof(struct node));
    if (temp == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    printf("\n Enter data to insert at beginning: ");
    scanf("%d", &temp->info);

    temp->next = start;
    temp->prev = NULL;

    if (start != NULL)
        start->prev = temp;

    start = temp;
    printf("Node inserted at the beginning.\n");
}

void insert_end() {
    struct node* temp;
    struct node* ptr;

    temp = (struct node*)malloc(sizeof(struct node));
    if (temp == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    printf("\n Enter data to insert at end: ");

```



```

scanf("%d", &temp->info);
temp->next = NULL;
temp->prev = NULL;

if (start == NULL) {
    start = temp;
} else {
    ptr = start;
    while (ptr->next != NULL) {
        ptr = ptr->next;
    }
    ptr->next = temp;
    temp->prev = ptr;
}
printf("Node inserted at the end.\n");
}

void insert_position() {
    struct node* temp;
    struct node* ptr;
    int pos, i;

    temp = (struct node*)malloc(sizeof(struct node));
    if (temp == NULL) {
        printf("Memory allocation failed\n");
        return;
    }

    printf("\n Enter data to insert: ");
    scanf("%d", &temp->info);
    temp->next = NULL;
    temp->prev = NULL;

    printf(" Enter position to insert node (starting from 1): ");
    scanf("%d", &pos);

    if (pos == 1) {
        temp->next = start;
        temp->prev = NULL;
        if (start != NULL)
            start->prev = temp;
        start = temp;
        printf("Node inserted at position 1.\n");
        return;
    }

```

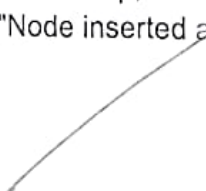
```
}

ptr = start;
for (i = 1; i < pos - 1 && ptr != NULL; i++) {
    ptr = ptr->next;
}

if (ptr == NULL) {
    printf("Position out of bounds.\n");
    free(temp);
} else {
    temp->next = ptr->next;
    temp->prev = ptr;

    if (ptr->next != NULL)
        ptr->next->prev = temp;

    ptr->next = temp;
    printf("Node inserted at position %d.\n", pos);
}
}
```



Experiment 6

* Doubly linked list

Q] Explain circular linked list

→ In a circular doubly linked list is a type of linked data structure consists of a set of sequentially linked records called nodes.

~~17/10~~

Experiment 7

* Stack

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MaxS
```

```
void push();
```

```
void pop();
```

```
void display();
```

```
int s[MaxS];
```

```
int top = -1;
```

```
int main() {
```

```
    int choice;
```

```
    printf("\n menu:");
```

```
    printf("\n 1. Push:");
```

```
    printf("\n 2. Pop:");
```

```
    printf("\n 3. Display\n");
```

```
    printf("\n 4. Exit\n");
```

```
    do {
```

```
        printf("\n Enter your choice:");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1: push();
```

```
            break;
```

```
            case 2: pop();
```

```
            break;
```

case 3: display();

break;

case 4: exit(0);

break;

}

} while (choice != 4);

return 0;

}

void push() {

int element;

if (top == max-1) {

printf("Overflow");

}

else {

printf("Enter value: ");

scanf("%d", &element);

top++;

arr[top] = element;

}

}

void pop() {

int item;

if (top == -1) {

printf("Under Flow");

}

else {

item = arr[top];

top--;


```

printf("The deleted element is: %d", item);
}
}

```

```

void display() {

```

```

    if (top == -1) {

```

```

        printf("Empty stack.");
    }

```

```

    else {

```

```

        printf("Stack element are: ");

```

```

        for (int i = 0; i <= top; i++) {

```

```

            printf("%d + ", sc[i]);

```

```

        }

```

```

    }

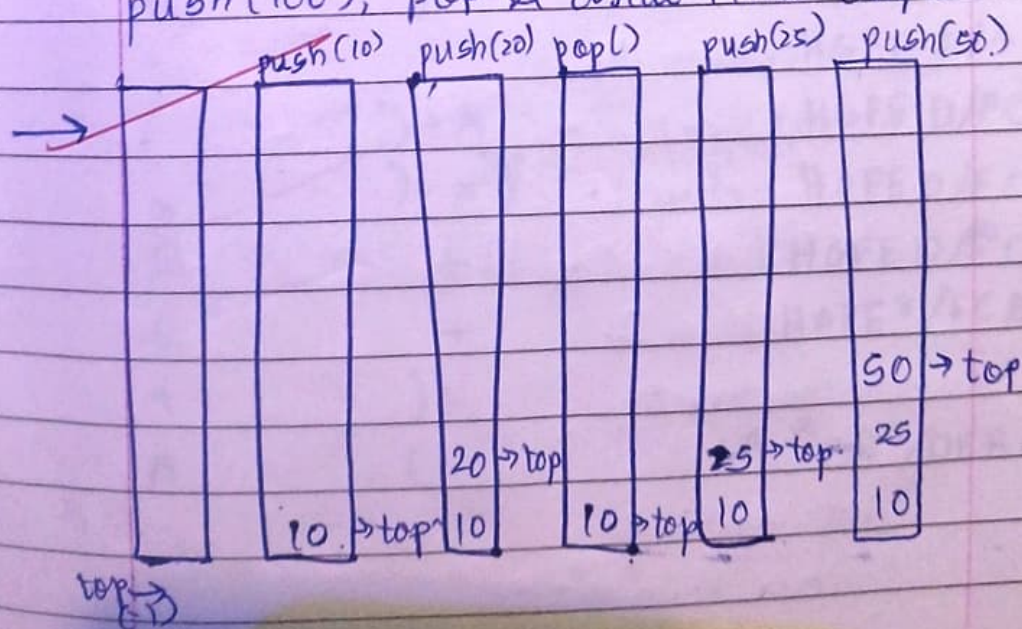
```

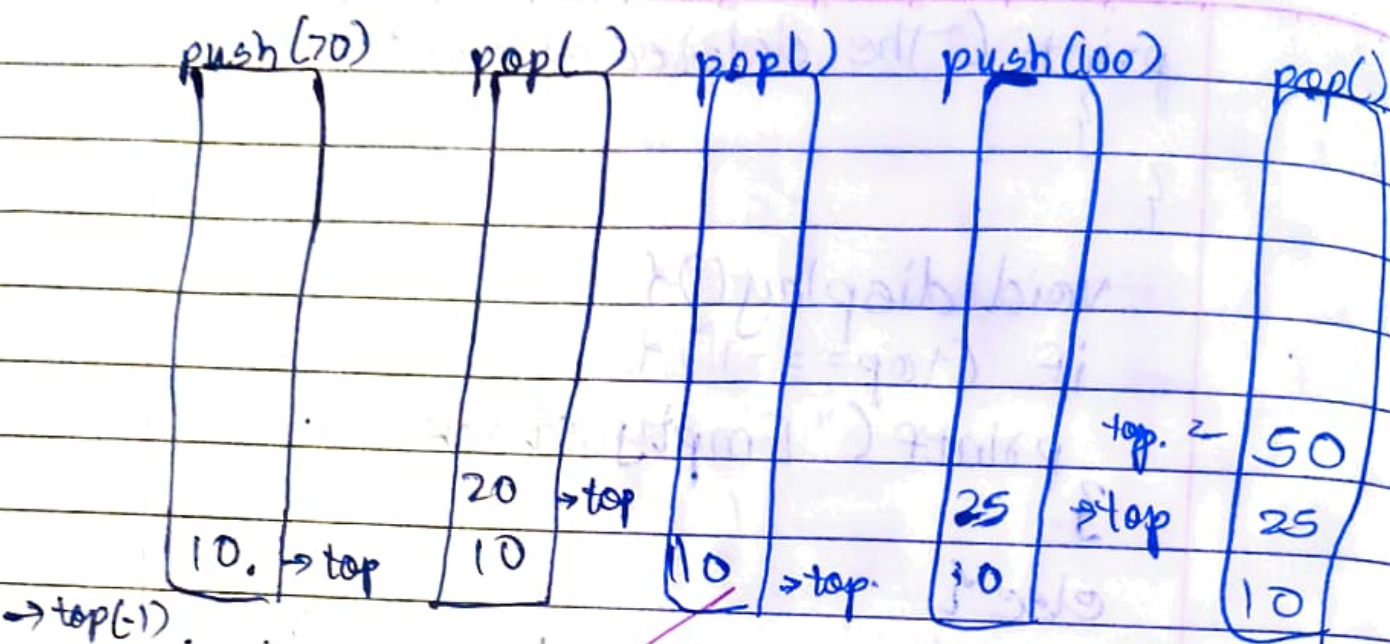
```

}

```

Q Stack size-8 for push(10), push(20), pop, push(25), push(50), push(70), pop, pop, push(100), pop & draw final output.





~~10/10~~

Experiment 8

Convert infix expression into prefix expression using stack.

Expression - $A + (B * C - (D / E ^ F) ^ G) H$

I/P	Stack	O/P
H	-	H
)	)	H
G	)	H G
*	) *	H G
)	) *	H G E
F	) *	H G E
^	) *) *	H G E F
E	) *) *	H G F E
/	) *) * /	H G F E ^ D
D	) *) * /	H G F E ^ D /
(	) * -	H G E E ^ D / ^
-	) * -	H G F F D / ^ C
(	) * -	H G F E D / ^ C B
x	) * - x	H G F E D / ^ C B
B	+	H G F E D / ^ C B
C	+	H G F E x / + C B ^ A +
+	) +	
A	(+	+ A - * C B ^ / D E F G H

i] Convert infix to postfix Expression.

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
char stack[100];
```

```
int top = -1;
```

```
void push (char x)
```

```
{ stack[++top] = x;
```

```
}
```

```
char pop()
```

```
{ if (top == -1)
```

```
return -1;
```

```
else
```

```
return stack[top--];
```

```
}
```

```
int priority (char x)
```

```
{ if (x == '(')
```

```
return 0;
```

```
if (x == '+' || x == '-')
```

```
return 1;
```

```
if (x == '*' || x == '/')
```

```
return 2;
```

```
return 0;
```

```
}
```

```
int main()
```

```
{ char exp[100];
```

```
char *e, x;
```

```
printf("Enter the expression");
```



```
scanf ("%s", &exp);  
printf ("\n");  
c = exp;
```

```
while (*e != "\0").
```

```
{ if (isalnum(*e))
```

```
printf ("%c", *e);
```

```
else if (*e == '(')
```

```
push(*e);
```

```
else if (*e == ')')
```

```
{
```

```
while ((x = pop()) != '(')
```

```
printf ("%c", x);
```

```
}
```

```
else
```

```
{
```

```
while (priority (stack[top]) >= priority)
```

```
printf ("%c", pop());
```

```
push(*e);
```

```
}
```

```
++;
```

```
}
```

```
while (top != -1)
```

```
{
```

```
printf ("%c", pop());
```

```
}
```

```
return;
```

ii] Write a C program to evaluate infix or postfix expression.

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#define size 40
```

```
int pop();
void push(int);
char postfix[size];
int stack[size], top = -1;
int main()
```

```
{
```

```
    int i, a, b, result, pval;
```

```
    char ch;
```

```
    for (i = 0; i < SIZE; i++)
```

```
    {
```

```
        stack[i] = -1;
```

```
    }
```

```
    printf("\n Enter a postfix expression!");
```

```
    scanf("%s", postfix);
```

```
    for (i = 0; postfix[i] != '\0'; i++)
```

```
    {
```

```
        ch = postfix[i];
```

```
        if (isdigit(ch))
```

```
        {
```

```
            push(ch - '0');
```

```
}
```

```
doe if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
```

```
{
```

```
    b = pop();
```

```
    a = pop();
```

```
    switch (ch)
```

```
{
```

```
    case '+': result = a + b;
```

```
        break;
```

```
    case '-': result = a - b;
```

```
        break;
```

```
    case '*': result = a * b;
```

```
        break;
```

```
    case '/': result = a / b;
```

```
        break;
```

```
}
```

```
    push(result);
```

```
}
```

```
}
```

```
pEval = pop();
```

```
printf("In The postfix evaluation is %d\n",
```

```
    pEval);
```

```
return 0;
```

```
}
```

```
void push (int n)
```

```
{
```

```
    if (top < size - 1)
```

```
{
```


stack[++top] = n;

}

else

{

printf("Stack is Full.\n");
exit(1);

}

}

int pop()

{

int n;

if (top > -1)

{

n = stack[top];

stack[top--] = -1;

return n;

}

else

{

printf("Stack is empty.\n");

exit(1);

}

}

* Algorithms:

1] Conversion from infix to postfix expression.

Step 1: Read Expression from left to right.

Step 2: If an operand (A-Z, 0-9) is encountered add it directly to output.

Step 3: If an operator is encountered.

Step 4: POP operator from stack to output if they have higher or equal precedence.

Step 5: Push the current operator onto the stack.

Step 6: If an opening parenthesis '(' is found, push it onto stack.

Step 7: If closing parenthesis ')' is found, pop operator from stack to output until opening parenthesis is encountered.

Step 8: After scanning entire expression, pop any remaining operators from stack to output.

2] Conversion from infix to prefix expression.

Step 1: Reverse the infix string. Note that while reversing, you must interchange left '(' and right ')' parentheses.

Step 2: Obtain the postfix expression of the given infix expression obtained from step 1.

Step 3: Reverse the obtained postfix expression (O/P) to get prefix expression.

3] Evaluating postfix expression:

Step 1: Scan the element of string from left to right.

Step 2: If the element is an operator POP two operands from the stack.

Step 3: If the element is an operand push it into the stack.

Step 4: The first pop is second operand & second pop is operand.

Step 5: Perform arithmetic operation & push result back into stack.

Step 6: When input expression is completed transversed pop operand stack & return the value.

~~RTIO~~

Experiment-9.

Q] Perform primitive operations on Linear Queue

```
→ #include <stdio.h>
#include <stdlib.h>
#define SIZE 5
void enqueue();
void dequeue();
void display();
int queue [SIZE];
int front = -1, rear = -1;
int main()
{
    int ch;
    printf("1. ENQUEUE\n
           2. DEQUEUE\n
           3. DISPLAY\n
           4. EXIT\n");
    do
    {
        printf("\n Enter your choice:");
        scanf("%d", &ch);
        switch(ch)
        {
            case 1: enqueue();
                    break;
            case 2: dequeue();
                    break;
```

PAGE No.
 DATE
 case 3: display();

break;

case 4: ~~dis~~exit(0);

default: printf("\n Invalid choice\n");

}

}

while (ch != 4)

return 0;

}

void enqueue()

{

int element;

if (rear == SIZE - 1)

{

printf("\n queue is full..");

}

else

{

printf("\n Enter Element to insert:");

scanf("%d", &element);

if (front == -1)

front = 0;

rear++

queue[rear] = element;

}

}

void dequeue()

{

printf

```
    printf("In Queue is Empty.");
```

```
else
```

```
{
```

```
    printf("In Deleted element %d", queue[Front]);  
    Front++;
```

```
}
```

```
}
```

```
void display()
```

```
{
```

```
    if (Front == -1 || Front > rear)
```

```
{
```

```
        printf("In Queue is empty...");
```

```
}
```

```
else
```

```
{
```

```
    printf("In Queue element are: \n");
```

```
    for (int i = Front; i <= rear; i++)
```

```
{
```

```
        printf("%d \n", queue[i]);
```

```
}
```

```
    printf("\n");
```

```
}
```

```
}
```


Q] Explain the concept of priority queue.

→ A priority queue is a data structure where each element has a priority & elements with higher priority are served before. Unlike regular queues (FIFO) priority queues process elements based on priority.

Types:

Ascending Priority Queue.

Descending Priority Queue.

Q] Write algorithm for insertion & deletion for linear queue.

→ Insertion:

- 1] Check for overflow condition
- 2] Check if the queue is empty. In case the queue is empty, the FRONT & REAR are set to zero, so that the new value can be stored at the 0th ~~lower~~ location, if the queue already has some values, then REAR is incremented so that it points to next location in the array.
- 3] The value is stored at the location pointed by REAR.
- 4] Exit.

Deletion:

1] We check for underflow condition. An underflow condition occurs if $\text{FRONT} = -1$ or $\text{FRONT} > \text{REAR}$. However, if queue has some values, then FRONT is incremented so that it now points to the next value in the queue.

2] Exit.

Q] Write algorithm for insertion & deletion for circular queue.

→ Deletion:

- 1] Check if queue is empty [$\text{FRONT} == -1$] = underflow.
- 2] Store the elements in front queue [FRONT]
- 3] If $\text{FRONT} == \text{REAR}$, set $\text{FRONT} = \text{REAR} = -1$
- 4] Exist.

Insertion:

- 1] Check if queue is Full [$(\text{REAR} - 1) / \text{size} = \text{FRONT}$] - overflow.
- 2] If queue is empty [$\text{FRONT} == -1$] set [$\text{FRONT} = \text{REAR} = 0$].
- 3] Go for REAR
- 4] Insert the new element at Queue [REAR]

Q1 Write application of queue.

→ Q1 Solving requests on a single server like a printer, CPU task scheduling.

Q2 Handling of interrupts in real-time. The interrupted are handled in the order as they arrive in that come first come first served.

Q3 FCFS process scheduling.

Q4 Online banking and requests.

Experiment-10.

Q- Write code for Circular Queue.

```
→ #include <stdio.h>
#include <stdlib.h>
#define SIZE 5

void enqueue();
void dequeue();
void display();
int queue [SIZE]
int front = -1, rear = -1;
int main() {
    int ch;
    printf("1.Enqueue \n 2.Dequeue \n 3.DISPLAY \n 4.Exit");
    do {
        printf("\n Enter your choice:");
        scanf("%d", &ch);
        switch (ch) {
            case 1: enqueue();
                    break;
            case 2: dequeue();
                    break;
            case 3: display();
                    break;
            case 4: exit(0);
        }
    } while (ch != 4);
}
```

PAGE No.
 DATE
 printf("Invalid choice!\n");
}

} while (ch != 4);

return 0;

}

void enqueue() {

int value;

if ((front == 0 & rear == SIZE - 1) || (rear + 1 == front))

{

printf("Queue is full\n");

}

else

{

printf("Enter value to insert: ");

scanf("%d", &value);

if (front == -1)

front = 0;

rear = (rear + 1) % SIZE;

queue[rear] = value;

printf("%d insert into queue", value);

}

}

void dequeue() {

if (front == -1)

{

printf("Queue is Empty");

}

else {

```
printf("In Delete %d From queue.", queue[front]);  
if (front == rear)
```

```
{
```

```
front = rear - 1;
```

```
}
```

```
else {
```

```
front = (front + 1) % SIZE;
```

```
}
```

```
}
```

```
}
```

```
void display() {
```

```
if (FRONT == -1)
```

```
{ printf("In Queue is Empty ! \n");  
}
```

```
else {
```

```
int i = front;
```

```
printf("In Queue element are : ");
```

```
while (1) {
```

```
printf(" %d ", queue[i]);
```

```
if (i == rear)
```

```
break;
```

```
i = (i + 1) % SIZE;
```

```
}
```

```
printf(" \n");
```

```
}
```


Write & explain application of priority queue.

- a. Traffic light control:
Manages signal priorities based on real-time traffic sensor data.
- b. Operating System Algorithm:
Used for scheduling process to execute high priority first.
- c. Huffman code:
Priority queue helps build Huffman trees for efficient data compression.

~~July 17/10~~

Experiment -11

Q1]

```
→ #include <stdio.h>
#include <stdlib.h>
struct node {
    int item;
    struct node *left;
    struct node *right;
};

struct node* createNode (int item) {
    struct node * newNode = (struct node*) malloc
    (size of (struct node));
    newNode -> item = item;
    newNode -> left = NULL;
    newNode -> right = NULL;
    return newNode;
}

void inorder (struct node* root)
    if (root == NULL)
        return;
    inorder (root -> left);
    printf ("%d", root -> item);
    inorder (root -> right);
}
```

```

void preorder(struct node* root){
    if (root == NULL)
        return;
    printf("%d", root->item);
    preorder(root->left);
    preorder(root->right);
}

```

```

void postorder(struct node* root){
    if (root == NULL)
        return;
    postorder(root->left);
    postorder(root->right);
    printf("%d", root->item);
}

```

```

int main(){

```

```

    struct node* root = createNode(40);

```

```

    insertAtLeft(root, 20);

```

```

    insertAtRight(root, 60);

```

```

    insertAtLeft(root->left, 10);

```

```

    insertAtRight(root->left, 30);

```

```

    insertAtLeft(root->right, 50);

```

```

    insertAtRight(root->right, 70);

```

```

    printf("In Inorder traversal:");

```

```

    inorder(root);

```

```

    printf("Preorder traversal:");

```

```

    preorder(root);

```

```

    printf("Postorder traversal:");

```

```

    postorder(root);

```

```

    return 0;
}

```



```
2] #include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int item;
```

```
    struct node * left;
```

```
    struct node * right;
```

```
};
```

```
struct node * createNode (int item) {
```

```
    struct node * newNode = (struct node *)
```

```
    malloc (size of (struct node));
```

```
    newNode -> item = item;
```

```
    newNode -> left = NULL;
```

```
    newNode -> right = NULL;
```

```
    return newNode;
```

```
}
```

```
struct node * insertAtLeft (struct node *  
root, int item)
```

```
{
```

```
    root -> left = createNode (item);
```

```
    return root -> right;
```

```
}
```

```
int main ()
```

```
{
```

```
    struct node * root = createNode (40);
```

```
    insertAtLeft (root, 20);
```

```
    insertAtRight (root, 60);
```

```
    insertAtLeft (root -> left, 10);
```

```
    insertAtRight (root -> left, 30);
```

```

insert At left (root → right, 50);
insert At right (root → right, 70);
return 0;
}

```

100/100

Experiment 12.

1. Matrix:

```
#include <stdio.h>
```

```
#define V 5
```

```
void init (int arr[V][V]) {
```

```
    int i, j;
```

```
    for (i = 0; i < V; i++) {
```

```
        for (j = 0; j < V; j++) {
```

```
            arr[i][j] = 0;
```

```
        }
```

```
    }
```

```
}
```

```
void insertEdge (int arr[V][V], int i, int j) {
```

```
    int i, j;
```

```
    for (i = 0; i < V; i++) {
```

```
        printf("%d", i);
```

```
        for (j = 0; j < V; j++) {
```

```
            printf("%d", arr[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
int main() {
```

```
    int adjMatrix[V][V];
```

```
    init (adjMatrix);
```



```

insert Edge (Adj Matrix, 0, 1);
insert Edge (Adj Matrix, 0, 2);
insert Edge (Adj Matrix, 0, 3);
insert Edge (Adj Matrix, 2, 0);
insert Edge ("", 2, 3);
insert Edge ("", 0, 3);
insert Edge ("", 0, 0);
insert Edge ("", 1, 3);
insert Edge ("", 4, 3);
insert Edge ("", 2, 4);

```

```

printf Adj Matrix (adj Matrix);
return 0;

```

Output:

0	1	1	1	0	0
1	1	0	1	0	0
2	1	1	0	1	1
3	1	1	1	0	0
4	0	0	1	1	0

2. linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define V 5
```

```
struct Node {
```

```
    int dest;
```

```
    struct Node *next;
```

```
};
```

```
struct Graph {
```

```
    struct Node *adj[V];
```

```
};
```

```
struct Node *newNode(int dest) {
```

```
    struct Node *temp = (struct Node *)
```

```
    malloc (size of (struct Node));
```

```
    temp->dest = dest;
```

```
    temp->next = NULL;
```

```
    return temp;
```

```
}
```

```
struct Graph * create_graph() {
```

```
    struct Graph * graph = (struct Graph *)
```

```
    malloc (size of (struct Graph));
```

```
    for (int i = 0; i < V; i++)
```

```
        graph->adj[i] = NULL;
```

```
    return graph;
```

```
}
```



```

void addEdge (struct Graph * graph,
int src, int dest) {
    struct Node * temp = newNode (dest);
    temp -> next = graph -> adj [src];
    graph -> adj [src] = temp;
    temp = newNode (src);
    temp -> next = graph -> adj [dest];
    graph -> adj [dest] = temp;
}

```

```

void print Graph (struct Graph * graph) {
    printf ("Adjacency list. \n");
    for (int i = 0; i < V; i++) {
        printf ("%d -> ", i);
        struct Node * temp = graph -> adj [i];
        while (temp) {
            printf ("%d ", temp -> dest);
            temp = temp -> next;
        }
    }
}

```

```

printf ("\n");
}

```

```

int main () {
    struct Graph * graph = create Graph ();
}

```




```

add Edge (graph, 0, 1);
add Edge (graph, 0, 2);
add Edge (graph, 0, 3);
add Edge (graph, 0, 4);
add Edge (graph, 1, 3);
add Edge (graph, 2, 3);
add Edge (graph, 2, 4);
add Edge (graph, 3, 4);

```

```

printGraph (graph);
return 0;
}

```

O/P Adjacent list

0 → 4 3 2 1

1 → 3 0

2 → 4 3 0

3 → 4 2 1 0

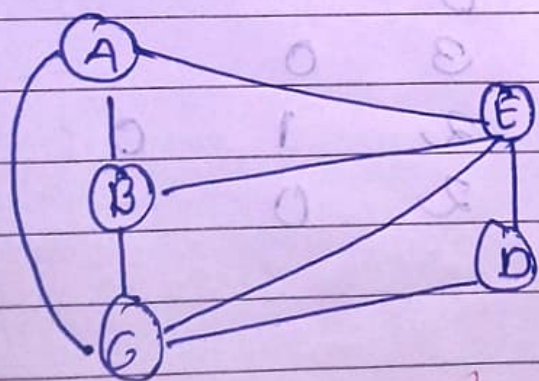
4 → 3 2 0

§

Q2] Explain Terminologies:

- i. Vertex: Each node of graph is represented as a vertex. In the examples, labeled circle is vertex.
- ii. Edge: Edge represents a path between 2 vertices or a line betⁿ 2 vertices.
- iii. Adjacency: 2 node or vertices are adjacent if they are connected to each other through an edge.
- iv. Path: Path represents a sequence of edges betⁿ the 2 vertices.

Q3]



	A	B	C	D	E
A	0	1	1	0	1
B	1	0	1	0	1
C	1	1	0	1	1
D	0	0	1	0	1
E	1	1	1	1	0

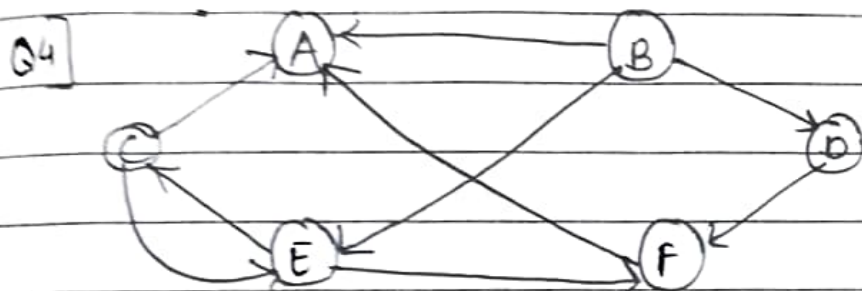
A → B → C → E

B → A → C → E

C → A → B → D → E

D → C → E

E → A → B → C → D



	A	B	C	D	E	F
A	0	0	0	0	0	0
B	1	0	0	1	1	0
C	1	0	0	0	1	0
D	0	0	0	0	0	1
E	0	0	1	0	0	1
F	1	0	0	0	0	0

A →

B → A → D → E

C → E → A

D → F

E → C → F

F → A